

**ADVANCED WEB PROGRAMMING
DATABASE MIGRATION**



KHEN MUHAMMAD CAHYO

244107023002

TI-21

**STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT**

PRACTICUM 1 (DATABASE SETTINGS)

- **Creating Database**



- **Change .env**

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=8889
DB_DATABASE=pwl-pos
DB_USERNAME=root
DB_PASSWORD=root
```

PRACTICUM 2 (MIGRATION)

- Creating Migration (Don't Have Relations and Have Relation)

```
0000_01_01_000000_create_levels_table.php
0001_01_01_000000_create_users_table.php
0001_01_01_000001_create_cache_table.php
0001_01_01_000002_create_jobs_table.php
2025_02_25_044129_create_categories_table.php
2025_02_25_044224_create_items_table.php
2025_02_25_044317_create_stocks_table.php
2025_02_25_044344_create_sales_table.php
2025_02_25_044354_create_sales_details_table.php
```

- Run Migration

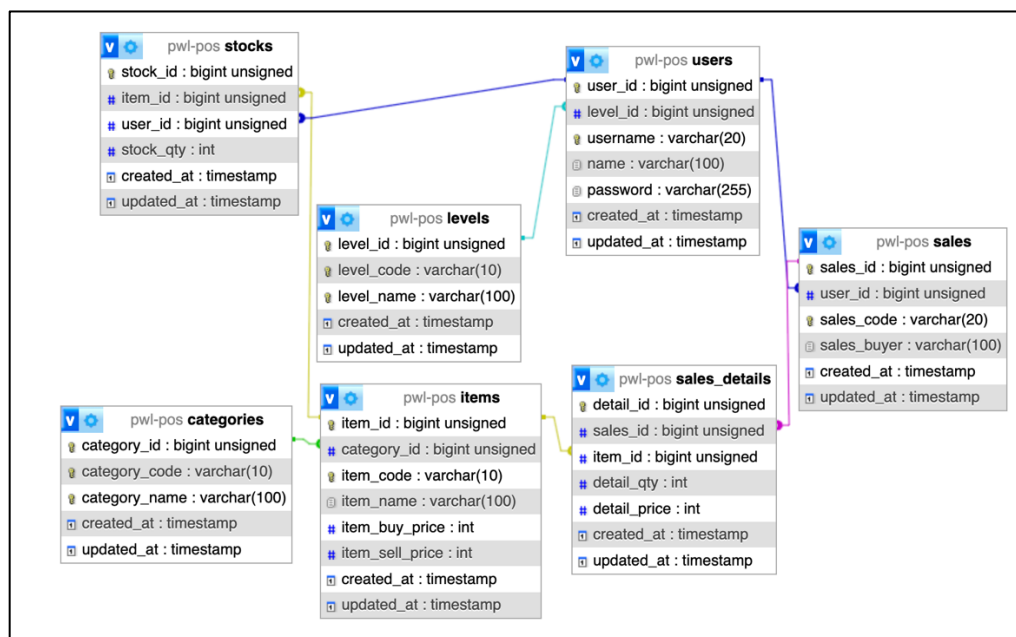
```
khencahyo@khens-MacBook-Air pwl-pos % php artisan migrate

WARN The database 'pwl-pos' does not exist on the 'mysql' connection.
Would you like to create it?
Yes

INFO Preparing database.
Creating migration table ..... 16.48ms DONE

INFO Running migrations.
0000_01_01_000000_create_levels_table ..... 20.44ms DONE
0001_01_01_000000_create_users_table ..... 30.62ms DONE
0001_01_01_000001_create_cache_table ..... 6.24ms DONE
0001_01_01_000002_create_jobs_table ..... 21.90ms DONE
2025_02_25_044129_create_categories_table ..... 10.38ms DONE
2025_02_25_044224_create_items_table ..... 15.66ms DONE
2025_02_25_044317_create_stocks_table ..... 25.51ms DONE
2025_02_25_044344_create_sales_table ..... 15.58ms DONE
2025_02_25_044354_create_sales_details_table ..... 25.59ms DONE
```

- PhpMyAdmin Table Designer



PRACTICUM 3 (SEEDER)

- **Creating Level & User Seeder**

```
class LevelSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['level_id' => '1', 'level_code' => 'ADM', 'level_name' => 'Administrator', 'created_at' =>
            now(), 'updated_at' => now()],
            ['level_id' => '2', 'level_code' => 'MNG', 'level_name' => 'Manager', 'created_at' => now
            (), 'updated_at' => now()],
            ['level_id' => '3', 'level_code' => 'STF', 'level_name' => 'Staff/Kasin', 'created_at' =>
            now(), 'updated_at' => now()],
        ];

        Level::insert($data);
    }
}
```

```
class UserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['level_id' => '1', 'username' => 'admin', 'name' => 'Administrator', 'password' => bcrypt
            ('admin')],
            ['level_id' => '2', 'username' => 'manager', 'name' => 'Manager', 'password' => bcrypt
            ('manager')],
            ['level_id' => '3', 'username' => 'staff', 'name' => 'Staff/Kasin', 'password' => bcrypt
            ('staff')],
        ];

        foreach ($data as $value) {
            User::create($value);
        }
    }
}
```

```
class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        // User::factory()->create([
        //     'name' => 'Test User',
        //     'email' => 'test@example.com',
        // ]);

        $this->call([
            LevelSeeder::class,
            UserSeeder::class,
        ]);
    }
}
```

- **Run Migration**

```
● khencahyo@Khens-MacBook-Air pwl-pos % php artisan db:seed --class=DatabaseSeeder

INFO Seeding database.
Database\Seeders\LevelSeeder ..... RUNNING
Database\Seeders\LevelSeeder ..... 17 ms DONE
Database\Seeders\UserSeeder ..... RUNNING
Database\Seeders\UserSeeder ..... 1,179 ms DONE
```

- **Creating Additional Migrations (Category, Items, Stock, Sales, Sales Details)**

```
class CategorySeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['category_id' => 1, 'category_code' => 'SNK1', 'category_name' => 'Sneaker Pria'],
            ['category_id' => 2, 'category_code' => 'SNK2', 'category_name' => 'Sneaker Wanita'],
            ['category_id' => 3, 'category_code' => 'OR1', 'category_name' => 'Sepatu Olahraga'],
            ['category_id' => 4, 'category_code' => 'CAS1', 'category_name' => 'Casual'],
            ['category_id' => 5, 'category_code' => 'SND1', 'category_name' => 'Sandal Pria'],
        ];

        Category::insert($data);
    }
}
```

```
class ItemSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['category_id' => 1, 'item_code' => 'SNK101', 'item_name' => 'Nike Air Max',
            'item_buy_price' => 500000, 'item_sell_price' => 750000],
            ['category_id' => 1, 'item_code' => 'SNK102', 'item_name' => 'Adidas Ultraboot',
            'item_buy_price' => 600000, 'item_sell_price' => 850000],
            ['category_id' => 2, 'item_code' => 'SNK201', 'item_name' => 'Puma RS-X', 'item_buy_price'
            => 550000, 'item_sell_price' => 800000],
            ['category_id' => 2, 'item_code' => 'SNK202', 'item_name' => 'Reebok Classic',
            'item_buy_price' => 400000, 'item_sell_price' => 650000],
            ['category_id' => 3, 'item_code' => 'OR101', 'item_name' => 'Asics Gel Kayano',
            'item_buy_price' => 700000, 'item_sell_price' => 1000000],
            ['category_id' => 3, 'item_code' => 'OR102', 'item_name' => 'Mizuno Wave Rider',
            'item_buy_price' => 650000, 'item_sell_price' => 950000],
            ['category_id' => 4, 'item_code' => 'CAS101', 'item_name' => 'Vans Old Skool',
            'item_buy_price' => 450000, 'item_sell_price' => 700000],
            ['category_id' => 4, 'item_code' => 'CAS102', 'item_name' => 'Converse Chuck Taylor',
            'item_buy_price' => 400000, 'item_sell_price' => 600000],
            ['category_id' => 5, 'item_code' => 'SND101', 'item_name' => 'Eiger Sandal Outdoor',
            'item_buy_price' => 300000, 'item_sell_price' => 500000],
            ['category_id' => 5, 'item_code' => 'SND102', 'item_name' => 'Skechers Sandal',
            'item_buy_price' => 350000, 'item_sell_price' => 550000],
        ];

        foreach ($data as $value) {
            Item::create($value);
        }
    }
}
```

```

class StockSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['item_id' => 1, 'user_id' => 3, 'stock_qty' => 10],
            ['item_id' => 2, 'user_id' => 3, 'stock_qty' => 5],
            ['item_id' => 3, 'user_id' => 3, 'stock_qty' => 20],
            ['item_id' => 4, 'user_id' => 3, 'stock_qty' => 15],
            ['item_id' => 5, 'user_id' => 3, 'stock_qty' => 30],
            ['item_id' => 6, 'user_id' => 3, 'stock_qty' => 20],
            ['item_id' => 7, 'user_id' => 3, 'stock_qty' => 10],
            ['item_id' => 8, 'user_id' => 3, 'stock_qty' => 10],
            ['item_id' => 9, 'user_id' => 3, 'stock_qty' => 10],
            ['item_id' => 10, 'user_id' => 3, 'stock_qty' => 5],
        ];

        foreach ($data as $value) {
            Stock::create($value);
        }
    }
}

```

```

class SalesSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['user_id' => 3, 'sales_code' => 'SAL1', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL2', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL3', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL4', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL5', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL6', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL7', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL8', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL9', 'sales_buyer' => fake()->name()],
            ['user_id' => 3, 'sales_code' => 'SAL10', 'sales_buyer' => fake()->name()],
        ];

        foreach ($data as $value) {
            Sales::create($value);
        }
    }
}

```

```

class SalesDetailSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            // Sales 1
            ['sales_id' => 1, 'item_id' => 1, 'detail_qty' => 2, 'detail_price' => 150000],
            ['sales_id' => 1, 'item_id' => 2, 'detail_qty' => 1, 'detail_price' => 200000],
            ['sales_id' => 1, 'item_id' => 3, 'detail_qty' => 3, 'detail_price' => 250000],

            // Sales 2
            ['sales_id' => 2, 'item_id' => 2, 'detail_qty' => 1, 'detail_price' => 200000],
            ['sales_id' => 2, 'item_id' => 4, 'detail_qty' => 2, 'detail_price' => 180000],
            ['sales_id' => 2, 'item_id' => 5, 'detail_qty' => 1, 'detail_price' => 120000],

            // Sales 3
            ['sales_id' => 3, 'item_id' => 1, 'detail_qty' => 1, 'detail_price' => 150000],
            ['sales_id' => 3, 'item_id' => 3, 'detail_qty' => 2, 'detail_price' => 250000],
            ['sales_id' => 3, 'item_id' => 5, 'detail_qty' => 1, 'detail_price' => 120000],

            // Sales 4
            ['sales_id' => 4, 'item_id' => 2, 'detail_qty' => 3, 'detail_price' => 200000],
            ['sales_id' => 4, 'item_id' => 4, 'detail_qty' => 2, 'detail_price' => 180000],
            ['sales_id' => 4, 'item_id' => 1, 'detail_qty' => 1, 'detail_price' => 150000],

            // Sales 5
            ['sales_id' => 5, 'item_id' => 3, 'detail_qty' => 2, 'detail_price' => 250000],
            ['sales_id' => 5, 'item_id' => 5, 'detail_qty' => 3, 'detail_price' => 120000],
            ['sales_id' => 5, 'item_id' => 1, 'detail_qty' => 1, 'detail_price' => 150000],

            // Sales 6
            ['sales_id' => 6, 'item_id' => 4, 'detail_qty' => 1, 'detail_price' => 180000],
            ['sales_id' => 6, 'item_id' => 2, 'detail_qty' => 2, 'detail_price' => 200000],
            ['sales_id' => 6, 'item_id' => 3, 'detail_qty' => 1, 'detail_price' => 250000],

            // Sales 7
            ['sales_id' => 7, 'item_id' => 5, 'detail_qty' => 1, 'detail_price' => 120000],
            ['sales_id' => 7, 'item_id' => 1, 'detail_qty' => 2, 'detail_price' => 150000],
            ['sales_id' => 7, 'item_id' => 4, 'detail_qty' => 3, 'detail_price' => 180000],

            // Sales 8
            ['sales_id' => 8, 'item_id' => 2, 'detail_qty' => 2, 'detail_price' => 200000],
            ['sales_id' => 8, 'item_id' => 3, 'detail_qty' => 1, 'detail_price' => 250000],
            ['sales_id' => 8, 'item_id' => 5, 'detail_qty' => 3, 'detail_price' => 120000],

            // Sales 9
            ['sales_id' => 9, 'item_id' => 4, 'detail_qty' => 1, 'detail_price' => 180000],
            ['sales_id' => 9, 'item_id' => 2, 'detail_qty' => 2, 'detail_price' => 200000],
            ['sales_id' => 9, 'item_id' => 1, 'detail_qty' => 3, 'detail_price' => 150000],

            // Sales 10
            ['sales_id' => 10, 'item_id' => 3, 'detail_qty' => 2, 'detail_price' => 250000],
            ['sales_id' => 10, 'item_id' => 5, 'detail_qty' => 1, 'detail_price' => 120000],
            ['sales_id' => 10, 'item_id' => 4, 'detail_qty' => 2, 'detail_price' => 180000],
        ];

        foreach ($data as $value) {
            SalesDetail::create($value);
        }
    }
}

```

```

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        // User::factory()->create([
        //     'name' => 'Test User',
        //     'email' => 'test@example.com',
        // ]);

        $this->call([
            LevelSeeder::class,
            UserSeeder::class,
            CategorySeeder::class,
            ItemSeeder::class,
            StockSeeder::class,
            SalesSeeder::class,
            SalesDetailSeeder::class,
        ]);
    }
}

```

● khencahyo@Khens-MacBook-Air: pwl-pos % php artisan db:seed --class=DatabaseSeeder

INFO Seeding database.

Database\Seeders\LevelSeeder	13 ms	RUNNING
Database\Seeders\LevelSeeder		DONE
Database\Seeders\UserSeeder	1,166 ms	RUNNING
Database\Seeders\UserSeeder		DONE
Database\Seeders\CategorySeeder	2 ms	RUNNING
Database\Seeders\CategorySeeder		DONE
Database\Seeders\ItemSeeder	9 ms	RUNNING
Database\Seeders\ItemSeeder		DONE
Database\Seeders\StockSeeder	7 ms	RUNNING
Database\Seeders\StockSeeder		DONE
Database\Seeders\SalesSeeder	21 ms	RUNNING
Database\Seeders\SalesSeeder		DONE
Database\Seeders\SalesDetailSeeder	15 ms	RUNNING
Database\Seeders\SalesDetailSeeder		DONE

PRACTICUM 4 (DB FACADE)

- **Creating Routes**

```
// Level Routes
Route::prefix('/level')->group(function () {
    Route::get('', [LevelController::class, 'index'])->name('level.index');
    Route::post('', [LevelController::class, 'store'])->name('level.store');
    Route::patch('/{id}', [LevelController::class, 'update'])->name('level.update');
    Route::delete('/{id}', [LevelController::class, 'delete'])->name('level.delete');
});
```

- **Creating Controller**

```
class LevelController extends Controller
{
    public function index() {
        $levels = DB::select('SELECT * FROM levels');

        return view('pages.level.index', compact('levels'));
    }

    public function store() {
        $operation = DB::insert('INSERT INTO levels(level_code, level_name) VALUES (?, ?)', [
            fake()->unique()->numerify('###'),
            fake()->name
        ]);

        if (!$operation) {
            return 'Failed to insert data!';
        }

        return 'Successfully insert data!';
    }

    public function update(string $id) {
        $operation = DB::update('UPDATE levels SET level_name = ? WHERE level_id = ?', ['Customer', $id]);

        if (!$operation) {
            return 'Failed to update data!';
        }

        return 'Successfully update data!';
    }

    public function delete(string $id) {
        $operation = DB::delete('DELETE FROM levels WHERE level_id = ?', [$id]);

        if (!$operation) {
            return 'Failed to delete data!';
        }

        return 'Successfully delete data!';
    }
}
```

- **Creating View**

```

/ken/MyCampus-Jobsheet/web-lanjut/week-3/pwl-pos/resources
<!-- lang="en" -->
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Customer Levels Data</title>
</head>
<body>
  <h1>Customer Levels Data</h1>
  <form action="{{ route('level.store') }}" method="POST">
    @csrf
    <button type="submit">Add New Level</button>
  </form>
  <table border="1" cellpadding="2" cellspacing="0">
    <thead>
      <tr>
        <th>ID</th>
        <th>Code</th>
        <th>Name</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      @foreach ($levels as $level)
        <tr>
          <td>{{ $level->level_id }}</td>
          <td>{{ $level->level_code }}</td>
          <td>{{ $level->level_name }}</td>
          <td>
            <form action="{{ route('level.delete', ['id' => $level->level_id]) }}" method="POST">
              @method('DELETE')
              @csrf
              <button type="submit">Delete</button>
            </form>
            <form action="{{ route('level.update', ['id' => $level->level_id]) }}" method="POST">
              @method('PATCH')
              @csrf
              <button type="submit">Update</button>
            </form>
          </td>
        </tr>
      @endforeach
    </tbody>
  </table>
</body>
</html>

```

- **Results**

Customer Levels Data

Add New Level

ID	Code	Name	Action
1	ADM	Administrator	Delete Update
2	MNG	Manager	Delete Update
3	STF	Staff/Kasir	Delete Update

PRACTICUM 5 (QUERY BUILDER)

- **Creating Routes**

```
// Category Routes
Route::prefix('/category')->group(function () {
    Route::get('/', [CategoryController::class, 'index'])->name('category.index');
    Route::post('/', [CategoryController::class, 'store'])->name('category.store');
    Route::patch('/{id}', [CategoryController::class, 'update'])->name('category.update');
    Route::delete('/{id}', [CategoryController::class, 'delete'])->name('category.delete');
});
```

- **Creating Controller**

```
class CategoryController extends Controller
{
    public function index() {
        $categories = DB::table('categories')->get();

        return view('pages.category.index', compact('categories'));
    }

    public function store() {
        $operation = DB::table('categories')->insert([
            'category_code' => fake()->unique()->numerify('###'),
            'category_name' => fake()->name()
        ]);

        if (!$operation) {
            return 'Failed to insert data!';
        }

        return 'Successfully insert data!';
    }

    public function update(string $id) {
        $operation = DB::table('categories')
            ->where('category_id', $id)
            ->update(['category_name' => 'Updated Category']);

        if (!$operation) {
            return 'Failed to update data!';
        }

        return 'Successfully update data!';
    }

    public function delete(string $id) {
        $operation = DB::table('categories')
            ->where('category_id', $id)
            ->delete();

        if (!$operation) {
            return 'Failed to delete data!';
        }

        return 'Successfully delete data!';
    }
}
```

- **Creating View**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Categories Data</title>
</head>
<body>
  <h1>Categories Data</h1>
  <form action="{{ route('category.store') }}" method="POST">
    @csrf
    <button type="submit">Add New Category</button>
  </form>
  <table border="1" cellpadding="2" cellspacing="0">
    <thead>
      <tr>
        <th>ID</th>
        <th>Code</th>
        <th>Name</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      @foreach ($categories as $category)
        <tr>
          <td>{{ $category->category_id }}</td>
          <td>{{ $category->category_code }}</td>
          <td>{{ $category->category_name }}</td>
          <td>
            <form action="{{ route('category.delete', ['id' => $category->category_id]) }}" method="POST">
              @method('DELETE')
              @csrf
              <button type="submit">Delete</button>
            </form>
            <form action="{{ route('category.update', ['id' => $category->category_id]) }}" method="POST">
              @method('PATCH')
              @csrf
              <button type="submit">Update</button>
            </form>
          </td>
        </tr>
      @endforeach
    </tbody>
  </table>
</body>
</html>
```

- **Results**

Categories Data

Add New Category

ID	Code	Name	Action
1	SNK1	Sneaker Pria	Delete Update
2	SNK2	Sneaker Wanita	Delete Update
3	OR1	Sepatu Olahraga	Delete Update
4	CAS1	Casual	Delete Update
5	SND1	Sandal Pria	Delete Update

PRACTICUM 6 (ELOQUENT ORM)

- **Creating Routes**

```
// User Routes
Route::prefix('/user')->group(function () {
    Route::get('', [UserController::class, 'index'])->name('user.index');
    Route::post('', [UserController::class, 'store'])->name('user.store');
    Route::patch('/{id}', [UserController::class, 'update'])->name('user.update');
    Route::delete('/{id}', [UserController::class, 'delete'])->name('user.delete');
});
```

- **Creating Controller**

```
class UserController extends Controller
{
    public function index() {
        $users = User::all();

        return view('pages.user.index', compact('users'));
    }

    public function store() {
        $operation = User::create([
            'level_id' => fake()->numberBetween(1, 3),
            'name' => fake()->name(),
            'username' => fake()->unique()->userName,
            'password' => bcrypt('123456'),
        ]);

        if (!$operation) {
            return 'Failed to insert data!';
        }

        return 'Successfully insert data!';
    }

    public function update(string $id) {
        $operation = User::where('user_id', $id)->update(['name' => fake()->name()]);

        if (!$operation) {
            return 'Failed to update data!';
        }

        return 'Successfully update data!';
    }

    public function delete(string $id) {
        $operation = User::destroy('user_id', $id);

        if (!$operation) {
            return 'Failed to delete data!';
        }

        return 'Successfully delete data!';
    }
}
```

- **Creating View**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Users Data</title>
</head>
<body>
  <h1>Users Data</h1>
  <form action="{{ route('user.store') }}" method="POST">
    @csrf
    <button type="submit">Add New User</button>
  </form>
  <table border="1" cellpadding="2" cellspacing="0">
    <thead>
      <tr>
        <th>ID</th>
        <th>Username</th>
        <th>Name</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      @foreach ($users as $user)
        <tr>
          <td>{{ $user->user_id }}</td>
          <td>{{ $user->username }}</td>
          <td>{{ $user->name }}</td>
          <td>
            <form action="{{ route('user.delete', ['id' => $user->user_id]) }}" method="POST">
              @method('DELETE')
              @csrf
              <button type="submit">Delete</button>
            </form>
            <form action="{{ route('user.update', ['id' => $user->user_id]) }}" method="POST">
              @method('PATCH')
              @csrf
              <button type="submit">Update</button>
            </form>
          </td>
        </tr>
      @endforeach
    </tbody>
  </table>
</body>
</html>
```

- **Results**

Users Data

Add New User

ID	Username	Name	Action
1	admin	Administrator	Delete Update
2	manager	Manager	Delete Update
3	staff	Staff/Kasir	Delete Update

QUESTION ANSWERS

1. Usually this key used as auth key in Laravel for encrypting and decrypting data, hashing password, etc that has a relationship with sensitive data.
2. Usually, app key generated automatically by Laravel when we install the project, but if we work with a team and we sharing the project, we must generate this key manually, and we can generate this key using artisan command **php artisan key:generate**.
3. By default, Laravel give us 3 migration file which is core migration for our Laravel project. Here is explanation for each migration file:
 - **users**: In this migration file, there are 3 table schemas. First one for table **users** that use for accommodate user account data and has a column for validate the account. Second one is **password_reset_tokens** for accommodate a reset token password that requested by user, and the last is **sessions** that used for accommodate user session in our web which created with Laravel.
 - **cache**: In this migration file, there are 2 table schemas. First one is **cache**, usually this table used for accommodate cached data in our Laravel app and this table only used if we set the cache driver in our Laravel app to **DB** or **Database**. Second one is **cache_locks**, same like **cache** but in this table the cached data has an owner. Usually we use this table if we want to caching data personally to user.
 - **jobs**: In this migration file, the are 3 table schemas. First one is **jobs**, this table used for accommodate queue data in our Laravel app, if we used Queue or Job feature in Laravel. Second one is **job_batches** to accommodate jobs ordering in our Laravel app. Last is **failed_jobs** that accommodates which jobs are failed in our Laravel app.
4. This builder will be make a timestamps column in our table as **created_at** and **updated_at**. Usually, this column used for tracking wheter user updating and creating the data.
5. Unsigned Big Integer and automatically the column that used this builder will be a primary key column in our table. Laravel provide us convenience for declare the primary key column in our table using it.
6. The different only in column name, if we only used **id()** by default Laravel will give default name in the column as **id**, but we can create custom column name for it using **id('level_id')** and then Laravel will give name in our primary column as **level_id**.

7. For ensure that the column has unique value. For example, when we create an account in other app, definitely we must insert a username right? For ensure that every username in users table are unique, we can use this builder **unique()**.
8. Because in users table, this table has reference to level table, and if we want to referencing the table as column, we must create a column with same data type in the referenced table, and in this case when we use **id()** builder as primary key column in our table, the data type for this column is Unsigned Big Integer. So, if we want to have references to the table that have primary key with **id()** builder, we must use UnsignedBigInteger in our referenced column.
9. Hash is a factory class that provided by Laravel that we can use for hashing our sensitive data, and in this case Hash is used for hashing user password data, and **Hash::make('1234')** syntax showed that we hash a 1234 password for user.
10. I think, the syntax like in PDO, when we used raw query, we must prepare the value that used for insert, update, or delete data in our table using **?**, and usually we used this syntax for avoiding SQL Injection in our app.
11. **protected \$table** is used for define which tables can use by Model, and **protected \$primaryKey** is used for define which primary key column in table that used by Model, by default the value of this property is **'id'**, so if we don't change the default primary key column name by Laravel, we don't need it.
12. For me, I like used ORM, because they provide me a convenience. Maybe in the case when we want to join other tables, we can just define the relationship in each table and the we only used **with()** to get the data for relations tables. But, in other case, I think I can used Query Builder also, maybe in case nested filtering data, I used this feature.